

TEMA N° 4



TEMA 4 INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS



EN ESTE TEMA SERÁS CAPAZ DE...

Para convertirte en un excelente Técnico en Sistemas Informáticos, necesitas herramientas que te permitan resolver problemas reales de manera ordenada y eficiente. En este tema, nos enfocaremos en dominar una de las metodologías más poderosas del desarrollo de software:

1. **Comprender** la diferencia fundamental entre pensar en código secuencial y pensar en "objetos" que interactúan, adoptando el paradigma más utilizado en la industria tecnológica.
2. **Modelar** problemas del mundo real de nuestra comunidad en Pailón, trasladando elementos físicos (como un estudiante, un aula o un sensor) a código estructurado mediante Clases y Objetos.



3. **Programar** soluciones robustas aplicando encapsulamiento, atributos y métodos, garantizando que los datos de tus aplicaciones estén seguros y organizados.



PRÁCTICA



¿Cómo organizamos el crecimiento tecnológico de nuestra propia casa de estudios?

Imagina la siguiente situación en nuestro Centro de Educación Alternativa (CEA) Herman Ortiz Camargo. Acabamos de inaugurar nuestro nuevo y bien equipado módulo educativo en el Barrio Saó. La dirección nos ha pedido a los estudiantes de Sistemas Informáticos que desarrollemos un sistema centralizado para gestionar absolutamente todo: desde el registro de los estudiantes que vienen del Barrio Central Fátima o San Expedito, hasta el control del inventario de las nuevas computadoras.

Hasta ahora, si tuvieras que programar esto, probablemente crearías listas enormes de variables sueltas: `nombre_estudiante_1`, `barrio_estudiante_1`, `computadora_marca_1`, `computadora_estado_1`. Si se inscriben 500 estudiantes de Las Misiones, San José y Ovidio Barberi, tu código se volvería un caos inmanejable de miles de líneas repetidas.

Si una computadora en el laboratorio necesita reparación, ¿cómo vinculas esa computadora específica con el estudiante exacto del Barrio 24 de Septiembre que la reportó, sin que los datos se mezclen accidentalmente con los del Barrio 27 de Mayo? Piensa por un momento: ¿No sería más fácil si en nuestro código pudiéramos crear una "plantilla" única llamada Estudiante y otra llamada Computadora, y simplemente hacer copias independientes de ellas con sus propios datos?



TEORÍA



4.1. PARADIGMA DE LA POO VS PROGRAMACIÓN ESTRUCTURADA

Como Técnico en Sistemas Informáticos, tu forma de pensar y estructurar soluciones es tu habilidad más valiosa. En programación, a estas formas de pensar las llamamos **paradigmas**.

La **Programación Estructurada** es el estilo clásico que probablemente ya conoces. Funciona como una receta de cocina: es una secuencia lógica de pasos (instrucciones, bucles for/while, y condicionales if/else) que se ejecutan de arriba hacia abajo. Es excelente para tareas sencillas, pero cuando el sistema crece, el código se vuelve un "código espagueti": un enredo de funciones y variables globales donde cambiar algo en la línea 10 puede romper algo en la línea 500.

Por otro lado, la **Programación Orientada a Objetos (POO)** es un paradigma que se basa en agrupar código y datos juntos en unidades lógicas llamadas "objetos". En lugar de pensar "¿Qué pasos debo seguir?", en POO piensas "¿Qué actores (objetos) participan en este problema y cómo se comunican entre sí?".



Si diseñamos el sistema del CEA, en programación estructurada tendríamos funciones



desconectadas procesando listas de datos. En POO, tendríamos un objeto Profesor, un objeto Estudiante y un objeto Aula, cada uno responsable de su propia información y acciones.

4.2. DEFINICIÓN DEL CONCEPTO DE CLASES Y OBJETOS

Para dominar la POO, debes entender a la perfección la relación entre dos conceptos hermanos: la Clase y el Objeto.

Una **Clase** es un molde, una plantilla o un plano de construcción. No es algo físico ni un dato real que puedas manipular; es simplemente la definición de lo que *debería* tener algo. Por ejemplo, el plano arquitectónico del nuevo módulo educativo del CEA en el Barrio Saó es una "Clase". El plano en sí no tiene paredes reales ni puedes dar clases dentro del papel, pero dicta exactamente cómo debe ser el edificio.

Un **Objeto** (también llamado *instancia*) es la materialización de esa Clase. Es el edificio ya construido en el Barrio Saó, con sus ladrillos, puertas y estudiantes adentro. A partir de una sola Clase (el molde), puedes crear múltiples Objetos independientes.

Si creamos una Clase llamada Deportista para el coliseo municipal del Barrio 27 de Mayo, los Objetos serían los jugadores reales: "Carlos", "María" y "José". Todos fueron creados con el mismo molde (Deportista), pero cada uno es único y tiene sus propios datos.

4.3. DECLARACIÓN DE ATRIBUTOS Y MÉTODOS

Cada Clase que diseñamos necesita saber cosas y hacer cosas. Aquí es donde entran los atributos y los métodos.

1. **Atributos:** Son las características, cualidades o datos que posee la clase. Se comportan como variables internas. Si nuestra Clase es Estudiante, sus atributos podrían ser nombre, edad, y barrio_residencia.
2. **Métodos:** Son las acciones que la clase puede realizar. Se programan exactamente igual que las funciones, pero viven dentro de la Clase. Para nuestro Estudiante, los métodos podrían ser estudiar(), presentar_examen() o registrar_asistencia().



DIAGRAMA DE CLASE: ESTUDIANTE



Veamos cómo se traduce esto a código usando Python:

Python

```
# Definición de la Clase usando la palabra reservada 'class'
class VecinoPailon:
    # Atributos de clase (comunes a todos)
    ciudad = "Pailón"
    # Método (acción) que puede realizar el objeto
    def saludar(self):
        # 'hace referencia al objeto mismo que está ejecutando la acción'
        print("¡Hola! Soy un vecino de Pailón y apoyo al CEA Herman Ortiz Camargo.")
```

4.4. PRINCIPIO DE ENCAPSULAMIENTO Y MODIFICADORES DE ACCESO (PUBLIC, PRIVATE)

Como Técnico, la seguridad de la información es tu responsabilidad. El **Encapsulamiento** es uno de los pilares de la POO. Consiste en ocultar los datos internos de un objeto para protegerlos de modificaciones accidentales o no autorizadas desde el exterior.

Imagina que estás desarrollando el sistema de pagos del comercio formal del Barrio Central Fátima. No querrías que cualquier otra parte del código pudiera acceder directamente al saldo de una tienda y cambiarlo a cero por error.



Para controlar esto, usamos **Modificadores de Acceso**:

1. **Público (Public):** Cualquier parte del programa puede ver y modificar el atributo o método. En Python, todos los atributos son públicos por defecto.
2. **Privado (Private):** Solo la propia Clase puede ver y modificar el atributo. Ningún código externo puede acceder a él directamente. En Python, esto se simula colocando dos guiones bajos antes del nombre del atributo (ejemplo: `__saldo`).

Si un dato es privado, la única forma de interactuar con él desde afuera es creando métodos públicos específicos llamados *getters* (para leer) y *setters* (para modificar bajo ciertas reglas), garantizando así un control total.

4.5. INSTANCIACIÓN DE OBJETOS Y USO DE CONSTRUCTORES BÁSICOS

La **Instanciación** es el momento mágico en el que tomamos nuestra Clase (el molde) y creamos un Objeto real en la memoria de la computadora.

Para que un objeto nazca con sus datos iniciales ya configurados (por ejemplo, que un estudiante nazca ya con su nombre y su barrio asignado), utilizamos un método especial llamado **Constructor**. En Python, este método se llama obligatoriamente `__init__`.





Python

```

class EstudianteCEA:    # Este es el método Constructor. Se ejecuta
                        # automáticamente al crear el objeto.
    def __init__(self, nombre_estudiante,
barrio_origen):        # Asignamos los parámetros recibidos a los atributos del
                        # objeto usando '
self
'
    self.nombre = nombre_estudiante    # Hacemos el barrio privado
usando '__
' para que no lo cambien por error    self.__barrio = barrio_origen
self.estado_matricula = "
Activo"    # Método público para ver el barrio protegido (Getter)
def
obtener_barrio(self):    return f"
El estudiante vive en el {
self.__barrio}
"

# ¡Instanciación! Creamos dos objetos reales a partir de la misma
Claseestudiante1 = EstudianteCEA("
Ana Gómez", "
Barrio Residencial Norte")
estudiante2 = EstudianteCEA("
Luis Pérez", "
Barrio Jardines de Pailón")
# Interactuamos con los objetos
print(estudiante1.nombre) # Salida: Ana
Gómez
print(estudiante2.obtener_barrio()) # Salida: El estudiante vive en el
Barrio Jardines de Pailón

```

4.6. LA POO COMO MOTOR DEL DESARROLLO WEB BACKEND

Como Técnico en Sistemas Informáticos, te enfrentarás a la creación de plataformas web dinámicas. Hoy en día, los frameworks más robustos del mercado (como Django en Python) operan bajo los principios de la POO.

Cuando necesites guardar información en una base de datos (PostgreSQL, MySQL), ya no escribirás complejas sentencias SQL a mano todo el tiempo. Utilizarás un ORM (Mapeo Objeto-Relacional). Un ORM toma tus Clases de Python y las convierte automáticamente en tablas de base de datos; los Atributos se convierten en las columnas, y cada Objeto instanciado se guarda como una fila nueva. Si creas la clase Proyecto, al instanciarla y guardarla, automáticamente se almacena en la nube.



4.7. INTEGRACIÓN DE POO CON INTERNET DE LAS COSAS (IOT) Y HARDWARE

La POO no se limita al software tradicional; es vital en la electrónica y el IoT. Si decides automatizar el sistema de iluminación del nuevo módulo del CEA utilizando microcontroladores (como el ESP32) y módulos de relés (relay), la POO te facilita la vida.

Puedes crear una clase llamada Rele. Sus atributos serían el pin_conexion y su estado (encendido/apagado). Sus métodos serían encender(), apagar() y programar_temporizador(). Así, si tienes un módulo de 8 canales, simplemente instancias 8 objetos independientes a partir de la misma clase Rele, manteniendo el código limpio, escalable y libre de errores al interactuar con el hardware real del aula.

4.8. INTELIGENCIA ARTIFICIAL Y MODELADO MODULAR DE CÓDIGO

La Inteligencia Artificial está transformando nuestra profesión. Asistentes de código como GitHub Copilot o ChatGPT comprenden el contexto mucho mejor cuando el código está orientado a objetos. Si le proporcionas a la IA una Clase bien definida con atributos encapsulados, la IA es capaz de generar métodos complejos, pruebas automáticas y expansiones de manera sumamente precisa. La IA "entiende" la estructura modular de las Clases, lo que te permite acelerar el desarrollo de proyectos complejos, delegando a la máquina la escritura de código repetitivo mientras tú, como Técnico, te concentras en el diseño lógico de la arquitectura.

CIERRE TEÓRICO

□ *Mitos vs. Realidades*

Mito Común	Realidad Tecnológica
"La POO hace que los programas sean mucho más lentos."	Si bien tiene una ligera carga por la gestión de memoria de los objetos, en el hardware actual y aplicaciones modernas, la diferencia es imperceptible. El beneficio en organización y reducción de errores supera ampliamente este detalle.
"Solo sirve para hacer aplicaciones visuales con botones."	Falso. La POO es el corazón de sistemas invisibles al usuario, como bases de datos, APIs de servidores backend, automatización



industrial, control de hardware y algoritmos de inteligencia artificial.

□ *Glosario de Términos*

1. **Paradigma:** Un modelo, estilo o enfoque fundamental utilizado para estructurar y resolver problemas en la programación.
2. **Instancia:** Sinónimo de Objeto. Es la manifestación física y real en memoria de una Clase teórica.
3. **Estado:** El conjunto de valores actuales que tienen los atributos de un objeto en un momento específico (ej. un estudiante en estado "Aprobado" o "Reprobado").
4. **Comportamiento:** Las acciones que un objeto es capaz de ejecutar, definidas por los métodos de su Clase.
5. **Abstracción:** La capacidad de ignorar los detalles complejos innecesarios de un problema y enfocarse solo en las características esenciales para modelarlo como una Clase.

□ *Ponte a Prueba*

1. ¿Qué es una Clase en el contexto de la Programación Orientada a Objetos?

1. a) Un archivo de base de datos.
2. b) Una función secuencial.
3. c) Un molde o plantilla para crear objetos.

2. ¿Cuál es el propósito del Encapsulamiento?

1. a) Hacer que el código se ejecute más rápido en la computadora.
2. b) Proteger los datos internos de un objeto impidiendo el acceso directo y no autorizado.
3. c) Conectar el programa con la base de datos de manera automática.

3. ¿Qué función cumple el constructor `__init__` en Python?

1. a) Elimina el objeto de la memoria cuando ya no se usa.



2. b) Instala las librerías necesarias en el servidor web.
3. c) Inicializa los atributos de un objeto en el momento exacto en que es creado o instanciado.



VALORACIÓN



Como futuros profesionales, la tecnología que desarrollamos impacta directamente a nuestra comunidad. Adoptar metodologías estructuradas como la POO no es solo un requerimiento técnico, es una **responsabilidad ética**.

Cuando programamos sistemas desordenados, aumentamos drásticamente el riesgo de vulnerabilidades de seguridad. Si los datos de los vecinos que usan los parques infantiles del Barrio Residencial Norte o la estación de tren en el Barrio 24 de Septiembre se ven comprometidos porque un programa no utilizó el *encapsulamiento* adecuado para proteger su privacidad, el responsable es el desarrollador.

Además, un código bien escrito mediante Clases es escalable y mantenible. Esto reduce la necesidad constante de reescribir software desde cero, lo que a gran escala en servidores globales minimiza el consumo energético innecesario. Un Técnico en Sistemas Informáticos que domina la POO garantiza que el software de instituciones locales, como nuestro CEA, pueda crecer durante años sin colapsar, democratizando el acceso seguro a la tecnología en Pailón.



PRODUCCIÓN



RETO FINAL: SISTEMA DE RESERVAS DEPORTIVAS

Es momento de ensuciarse las manos y programar. El Coliseo Municipal ubicado en el Barrio 27 de Mayo necesita un sistema básico en Python para gestionar las reservas de las canchas. Diseñarás la estructura utilizando la Programación Orientada a Objetos.

Paso 1: Abre tu editor de código (como VS Code) y crea un archivo llamado `reservas_coliseo.py`.

Paso 2: Define la clase `CanchaDeportiva`.



Paso 3: Crea el constructor para asignarle un nombre a la cancha, el deporte y establecer su estado de reserva como privado.

Paso 4: Desarrolla los métodos para reservar y liberar la cancha.

A continuación, tienes la base del laboratorio. Completa tu código, ejecútalo y verifica la consola:

Python

```
# 1. Definimos La Clase principal
class CanchaDeportiva:
    # 2. Constructor para inicializar cada nueva cancha
    def __init__(self, nombre_cancha, tipo_deporte):
        self.nombre = nombre_cancha
        self.deporte = tipo_deporte
        # Atributo PRIVADO para proteger el estado de La reserva
        self.__esta_reservada = False
        # 3. Método para intentar reservar La cancha
        def reservar(self, vecino_responsable):
            # Verificamos si NO está reservada
            if not self.__esta_reservada:
                self.__esta_reservada = True # Cambiamos el estado a reservado
                print(f"Éxito: La {self.nombre} ha sido reservada para {self.deporte} por el vecino {vecino_responsable}."
                    else:
                        print(f"Error: La {self.nombre} ya se encuentra ocupada actualmente.")
            # 4. Método para liberar la cancha cuando termina el partido
            def liberar(self):
                self.__esta_reservada = False
                print(f"Información: La {self.nombre} ahora está libre y disponible.")
            # -
            -- PRUEBA DEL SISTEMA (Instanciación) -
            --# Creamos dos objetos (canchas reales del coliseo)
            cancha_principal = CanchaDeportiva("Cancha Principal 27 de Mayo", "Futsal")
            cancha_auxiliar = CanchaDeportiva("Cancha Auxiliar", "Básquetbol")
            # Simulamos la interacción de los vecinos
            cancha_principal.reservar("Carlos de Barrio María Belén")
            # Intentamos reservar la misma cancha otra vez (debe dar error)
            cancha_principal.reservar("Luis de Barrio San Antonio")
            # Reservamos la otra cancha disponible
            cancha_auxiliar.reservar("Luis de Barrio San Antonio")
```



```
Ana de Barrio Oto Waver")
```

```
# Finaliza el partido y liberamos la cancha principalcancha_principal.liberar()
```

5. RECURSOS ADICIONALES

Para consolidar tu perfil como Técnico en Sistemas Informáticos y profundizar tus conocimientos en POO, te recomiendo explorar las siguientes fuentes de alto valor académico y profesional:

1. **Documentación Oficial de Python (Sección de Clases):** La guía definitiva y más actualizada para entender a nivel profundo cómo Python maneja los objetos bajo el capó.
 1. *Búsqueda recomendada:* "Python Official Documentation Classes".
2. **Mozilla Developer Network (MDN Web Docs) - Object-Oriented JavaScript:** Si decides llevar este conocimiento hacia el desarrollo web frontend, MDN explica magistralmente cómo se aplica la POO en los navegadores.
 1. *Búsqueda recomendada:* "MDN POO JavaScript introducción".
3. **Repositorios Abiertos en GitHub:** La mejor manera de aprender a estructurar Clases es leyendo código de profesionales. Explora repositorios públicos de pequeños proyectos (como calculadoras o sistemas de inventario simples en Python) para ver cómo los expertos organizan sus atributos y métodos.
 1. *Búsqueda recomendada:* "GitHub Python simple OOP examples".